TurtleND: A Turtle That Carries Its Own Orthonormal Frame

Frame-Based Navigation in Three and N Dimensions
by Givens Rotations

Eric G. Suchanek, PhD
suchanek@flux-frontiers.com
Flux-Frontiers
https://github.com/Flux-Frontiers/turtlend

October 8, 2026

Abstract

A turtle is a moving point that carries its own coordinate system. It moves along its heading and rotates its frame; it never refers to global axes. This article describes `TurtleND`, a turtle that carries an orthonormal frame of N basis vectors through $\mathbb{R}^N$ and rotates by Givens rotations in the plane of any two of them. The three-dimensional turtle it generalizes, `Turtle3D`, was written in C in 1990 for building protein structures from bond lengths, bond angles and dihedral angles. The N-dimensional version was written for navigating learned embedding spaces. We give the state, the operations and their definitions, the invariants they preserve, the conventions that make `TurtleND`(3) agree with `Turtle3D` on every command, and the algorithms for orienting the frame to arbitrary directions, re-orthonormalizing it, and adding a dimension to a live turtle. The implementation is a NumPy package, `turtlend`, of about five hundred lines, with no other dependencies.

Contents

1 Introduction **3**

2 Mathematical Preliminaries **3**

3 The TurtleND **4**
3.1 State and Conventions 4
3.2 Operations 4
3.3 Invariants 5
3.4 The Local Coordinate System 5

4 Agreement with Turtle3D **6**

5 Frame Construction Algorithms **7**
5.1 Orient 7
5.2 Orient Toward 7
5.3 Orthonormalize 8
5.4 Expand Dimension 8

6 Numerical Behavior **9**
6.1 Cost . . . 9
6.2 Drift . . . 9
6.3 Determinism . . . 9

7 Applications **10**
7.1 Building Molecules . . . 10
7.2 Navigating an Embedding Space . . . 10
7.3 Adding a Time Axis . . . 10

8 Implementation **11**

9 Conclusion **11**

1 Introduction

Turtle geometry, as Papert and later Abelson and diSessa presented it, replaces the Cartesian description of a path with an intrinsic one [1, 6]. The turtle knows where it is and which way it faces. It moves forward some distance and turns through some angle, and a curve is the record of those commands. The description is local: it does not depend on where the origin is or how the axes are drawn.

The three-dimensional turtle adds a second and third direction to the heading. With a heading, a left and an up vector, the turtle has a complete right-handed frame, and the commands become Move, Turn, Pitch and Roll. This is exactly the description a chemist uses for a molecule. A bond has a length, the angle between two bonds is a bond angle, and the rotation about a bond is a dihedral. A turtle placed on an atom can walk to the next atom by one move, one yaw and one roll, with no reference to global coordinates at all. I wrote such a turtle in C in 1990 for the Proteus modeling program [5], ported it to Python for proteusPy [7, 8], and it has built thousands of disulfide bonds since.

The generalization to N dimensions came from a different problem. The WaveRider project [10] navigates the embedding spaces produced by neural networks, where N is in the hundreds or thousands, and where the data concentrate on a low-dimensional manifold whose tangent space changes from place to place. A turtle that carries N orthonormal basis vectors, and that can align them to the local principal directions of the data, gives a moving coordinate system for such a space. The three-dimensional rotations become instances of one primitive, a Givens rotation in the plane of two basis vectors [2].

Both turtles lived as duplicate copies inside proteusPy and WaveRider until September 2026, when they were moved into a single package, `turtlend` [9]. This article is the description of that package. It covers the mathematics of the frame and its rotations (Section 2), the `TurtleND` state and operations (Section 3), the conventions that make it agree with `Turtle3D` (Section 4), the frame-construction algorithms (Section 5), the numerical behavior (Section 6), and the two applications that motivated it (Section 7). It does not cover the manifold navigation built on top of the turtle in WaveRider; that is documented with WaveRider itself.

2 Mathematical Preliminaries

Definition 2.1 (Orthonormal Frame)**.** *An* orthonormal frame *in* $\mathbb{R}^N$ *is an ordered set of* N *vectors* $\mathbf{F} = (\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{N-1})$ *with*

$$\langle \mathbf{b}_i, \mathbf{b}_j \rangle = \delta_{ij} \qquad \forall\, i, j \in \{0, \ldots, N-1\}, \tag{1}$$

where δ_{ij} *is the Kronecker delta. Equivalently, the matrix* $F \in \mathbb{R}^{N \times N}$ *whose rows are the* $\mathbf{b}_i$ *satisfies* $FF^\top = F^\top F = \mathbf{I}_N$*. We write* $\mathbf{e}_i$ *for the* i*-th standard basis vector of* $\mathbb{R}^N$*, so that the identity frame is* $\mathbf{F} = (\mathbf{e}_0, \ldots, \mathbf{e}_{N-1})$ *and* $F = \mathbf{I}_N$*.*

The row convention matters for what follows. With basis vectors as rows, the product $F\mathbf{v}$ is the vector of inner products $\langle \mathbf{b}_i, \mathbf{v} \rangle$, which are the coordinates of $\mathbf{v}$ in the frame, and $F^\top \boldsymbol{\ell}$ is the linear combination $\sum_i \ell_i \mathbf{b}_i$, which maps frame coordinates back to $\mathbb{R}^N$.

Definition 2.2 (Givens Rotation)**.** *A* Givens rotation $G(i, j, \theta)$ *in* $\mathbb{R}^N$ *is the identity outside the coordinate plane* (i, j) *and a plane rotation inside it [2, 3]:*

$$G(i, j, \theta) = \mathbf{I}_N + (\cos\theta - 1)\,(\mathbf{e}_i\mathbf{e}_i^\top + \mathbf{e}_j\mathbf{e}_j^\top) + \sin\theta\,(\mathbf{e}_i\mathbf{e}_j^\top - \mathbf{e}_j\mathbf{e}_i^\top). \tag{2}$$

Its only entries that differ from the identity are $G_{ii} = G_{jj} = \cos\theta$*,* $G_{ij} = \sin\theta$ *and* $G_{ji} = -\sin\theta$*.*

A Givens rotation is orthogonal, $GG^\top = \mathbf{I}_N$, and has determinant $+1$. Givens introduced these rotations to reduce a matrix to triangular form one plane at a time; the turtle uses them for the opposite purpose, to rotate a frame one plane at a time while leaving every other basis vector untouched.

Proposition 2.1 (Frame Rotation)**.** *If F is the row matrix of an orthonormal frame, then $F' = G(i, j, \theta)\, F$ is the row matrix of an orthonormal frame, and its rows are*

$$\mathbf{b}'_i = \cos\theta\, \mathbf{b}_i + \sin\theta\, \mathbf{b}_j, \qquad \mathbf{b}'_j = \cos\theta\, \mathbf{b}_j - \sin\theta\, \mathbf{b}_i, \qquad \mathbf{b}'_k = \mathbf{b}_k \;\; (k \neq i, j). \tag{3}$$

Proof. $F'F'^\top = GFF^\top G^\top = GG^\top = \mathbf{I}_N$. Rows i and j of GF are the stated combinations by Equation (2); every other row of G is a row of the identity. □

Equation (3) is the whole of the rotation machinery. It says that $\mathbf{b}_i$ rotates toward $\mathbf{b}_j$ by θ within the plane the two span, $\mathbf{b}_j$ follows it around, and nothing else moves. In three dimensions, a rotation in the plane of two basis vectors is a rotation about the third, which is why the named three-dimensional rotations are special cases.

3 The TurtleND

3.1 State and Conventions

A `TurtleND` in $N \geq 2$ dimensions holds a position $\mathbf{p} \in \mathbb{R}^N$ and an orthonormal frame $\mathbf{F}$ stored as the rows of an $N \times N$ matrix. The basis vectors are named by index (Table 1). A new turtle sits at the origin with the identity frame: $\mathbf{p} = \mathbf{0}$, $F = \mathbf{I}_N$. The `position`, `frame`, `heading`, `left`, `up` and `basis(i)` accessors return copies, so the only way to change the turtle is through its methods.

Table 1: Frame conventions. The names are inherited from `Turtle3D`.

Row	Name	Role
$\mathbf{b}_0$	heading	direction of MOVE
$\mathbf{b}_1$	left	
$\mathbf{b}_2$	up	requires $N \geq 3$
$\mathbf{b}_3, \ldots, \mathbf{b}_{N-1}$	unnamed	reached through `basis(i)` and `rotate`

The turtle also carries a pen flag and an optional tape. When `recording` is set, every MOVE appends the new position to the tape, so a path can be recovered after the fact. The pen is kept for compatibility with `Turtle3D` and has no effect on the geometry.

3.2 Operations

Table 2 lists the operations and their definitions. All angles in the public interface are in degrees; the implementation converts to radians once, at the entry to `rotate`. A rotation never changes the position and a move never changes the frame.

ROTATE is the one primitive. TURN, ROLL, PITCH and YAW are calls to it with fixed plane indices and, for PITCH and YAW, a transformed angle. The signs and the 180° offset are not arbitrary; they are the conventions of `Turtle3D`, and Section 4 explains where they come from. ROLL and PITCH require $N \geq 3$ because they involve $\mathbf{b}_2$. ROTATE rejects $i = j$ and indices outside $[0, N)$.

Table 2: TurtleND operations. G is the Givens rotation of Definition 2.2, applied to the frame as in Proposition 2.1.

Operation	Definition	Plane
Move(d)	$\mathbf{p} \leftarrow \mathbf{p} + d\,\mathbf{b}_0$	
Rotate(θ, i, j)	$F \leftarrow G(i, j, \theta)\,F$	$(\mathbf{b}_i, \mathbf{b}_j)$
Turn(θ)	Rotate($\theta, 0, 1$)	heading, left
Roll(θ)	Rotate($\theta, 1, 2$)	left, up
Pitch(θ)	Rotate($-\theta, 0, 2$)	heading, up
Yaw(θ)	Rotate($180° - \theta, 0, 1$)	heading, left
ToLocal($\mathbf{g}$)	$\boldsymbol{\ell} = F\,(\mathbf{g} - \mathbf{p})$	
ToGlobal($\boldsymbol{\ell}$)	$\mathbf{g} = \mathbf{p} + F^\top \boldsymbol{\ell}$	
Orthonormalize()	$F \leftarrow$ sign-preserving Q of $F^\top = QR$	
Orient($\mathbf{p}, \mathbf{h}, \mathbf{l}$)	Algorithm 1	
OrientToward($\mathbf{d}$)	Algorithm 2	(heading, $\mathbf{d}$)
ExpandDim()	Algorithm 3	

3.3 Invariants

Proposition 3.1 (Frame Invariant)**.** Rotate*, and therefore every named rotation, maps an orthonormal frame to an orthonormal frame.*

Proof. Proposition 2.1. □

Proposition 3.2 (Coordinate Transform Invertibility)**.** *For every* $\mathbf{g} \in \mathbb{R}^N$ *and every* $\boldsymbol{\ell} \in \mathbb{R}^N$*,*

$$\text{ToGlobal}(\text{ToLocal}(\mathbf{g})) = \mathbf{g} \qquad \textit{and} \qquad \text{ToLocal}(\text{ToGlobal}(\boldsymbol{\ell})) = \boldsymbol{\ell}.$$

Proof. Let $\boldsymbol{\delta} = \mathbf{g} - \mathbf{p}$. Then $\text{ToGlobal}(F\boldsymbol{\delta}) = \mathbf{p} + F^\top F\boldsymbol{\delta} = \mathbf{p} + \boldsymbol{\delta} = \mathbf{g}$, and $\text{ToLocal}(\mathbf{p} + F^\top\boldsymbol{\ell}) = FF^\top\boldsymbol{\ell} = \boldsymbol{\ell}$, since $F^\top F = FF^\top = \mathbf{I}_N$. □

Proposition 3.3 (Move Commutes with the Frame)**.** Move(d) *followed by* Rotate *gives the same position as* Rotate *followed by* Move(d) *if and only if the rotation does not involve* $\mathbf{b}_0$*.*

Proof. Move adds $d\,\mathbf{b}_0$ to the position. A rotation in a plane not containing index 0 leaves $\mathbf{b}_0$ fixed, so the two orders add the same vector. A rotation in a plane containing index 0 changes $\mathbf{b}_0$, so the two orders add different vectors unless $d = 0$ or θ is a multiple of 360°. □

Proposition 3.3 is why the turtle's path is a sequence of straight segments joined at the positions where a heading rotation occurred. Roll never changes the path in any dimension; Turn, Pitch and Yaw always do.

3.4 The Local Coordinate System

The frame gives the turtle a moving coordinate system centered on its position. ToLocal expresses a global point in that system, one inner product per basis vector, and ToGlobal is the inverse (Proposition 3.2). Both are single matrix-vector products and cost $O(N^2)$.

Two uses recur. In molecular building, the turtle is placed on an atom with its frame aligned to the local bond geometry, and the coordinates of nearby atoms are read out in the turtle's frame. The same atom positions expressed this way are the same for every residue of the same type, regardless

of where the residue sits in the protein; this is what lets a residue template be built once and placed anywhere. In manifold navigation, a gradient or a displacement is taken into the frame with ToLocal, its components are modified independently (kept, scaled or zeroed), and the result is taken back out with ToGlobal [10]. The frame is the device that makes "the component along the second principal direction" a meaningful phrase.

4 Agreement with Turtle3D

`Turtle3D` stores three named vectors, heading $\mathbf{h}$, left $\mathbf{l}$ and up $\mathbf{u}$, and implements each rotation as an explicit pair of component updates. Its conventions are the ones that were convenient for building molecules in 1990, and `TurtleND` adopts them so that the two turtles are interchangeable in three dimensions. Table 3 lists the four rotations as `Turtle3D` writes them and the Givens plane and angle that reproduce each.

Table 3: The `Turtle3D` rotations and their `TurtleND` equivalents. $c = \cos\alpha$ and $s = \sin\alpha$ for the angle α given in the third column.

Turtle3D	**Update**	**Angle α**	**TurtleND**
`turn(a)`	$\mathbf{h}' = c\mathbf{h} + s\mathbf{l}$, $\mathbf{l}' = c\mathbf{l} - s\mathbf{h}$	a	`rotate(a, 0, 1)`
`roll(a)`	$\mathbf{l}' = c\mathbf{l} + s\mathbf{u}$, $\mathbf{u}' = c\mathbf{u} - s\mathbf{l}$	a	`rotate(a, 1, 2)`
`pitch(a)`	$\mathbf{h}' = c\mathbf{h} - s\mathbf{u}$, $\mathbf{u}' = c\mathbf{u} + s\mathbf{h}$	a	`rotate(-a, 0, 2)`
`yaw(a)`	$\mathbf{h}' = c\mathbf{h} + s\mathbf{l}$, $\mathbf{l}' = c\mathbf{l} - s\mathbf{h}$	$180^\circ - a$	`rotate(180 - a, 0, 1)`

Three of the four follow directly from Equation (3). `turn` and `roll` are rotations of the first index toward the second by the given angle. `pitch` rotates the heading toward $-\mathbf{u}$, which is a rotation of $\mathbf{b}_0$ toward $\mathbf{b}_2$ by $-a$. The fourth, `yaw`, is a turn by the supplement of the angle, and it exists because of how bond angles are defined.

Remark 4.1 (Why yaw is $180^\circ - \theta$)**.** *Place the turtle on atom A heading toward atom B, and move the bond length so that it stands on B. Its heading is now the direction $A \to B$. The bond angle θ_{ABC} is the angle at B between the bonds $B \to A$ and $B \to C$, and $B \to A$ is the reverse of the heading. To point the heading along $B \to C$, the turtle must turn by $180^\circ - \theta_{ABC}$. `yaw` takes the bond angle from the chemistry tables and does the subtraction, so that building an atom is `move(length)`, `yaw(angle)`, `roll(dihedral)` with the tabulated values and nothing else. Users who want a plain turn by θ call `turn`.*

Theorem 4.1 (Equivalence in Three Dimensions)**.** *For every finite sequence of* Move, Turn, Roll, Pitch *and* Yaw *commands with the same arguments, `TurtleND`(3) started at the identity frame and `Turtle3D` started at its default frame end at the same position with the same heading, left and up vectors, up to floating-point rounding.*

Proof. Both turtles start at $\mathbf{p} = \mathbf{0}$ with $(\mathbf{h}, \mathbf{l}, \mathbf{u}) = (\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2)$. Move is the same update in both. For each rotation, Table 3 shows that the `Turtle3D` component updates are Equation (3) for the stated plane and angle, which is what `TurtleND` computes. Both implementations renormalize the two changed vectors after each rotation, and renormalization of vectors that are already unit length up to rounding changes them only by rounding. By induction on the length of the sequence the states agree at every step. □

The test suite of `turtlend` checks this theorem over a random sequence of 200 commands drawn from all five operations with arguments uniform in $[-180, 180]$, comparing position, frame and both coordinate transforms at the end of the sequence to a tolerance of 10^{-7}, and separately on every individual operation.

Remark 4.2 (Handedness of ORIENT)**.** *The theorem is stated for the five movement commands and not for* ORIENT*. `Turtle3D.orient` completes the frame with a cross product, $\mathbf{u} = \mathbf{h} \times \mathbf{l}$, and is always right-handed. `TurtleND.orient` completes it by Gram-Schmidt against the standard basis vectors (Algorithm 1), which has no notion of handedness in general N, and in three dimensions can produce $\mathbf{b}_2 = -\mathbf{h} \times \mathbf{l}$. The frame is orthonormal either way. A caller who needs the right-handed completion in three dimensions should use `Turtle3D`, or check the sign of $\langle \mathbf{b}_2, \mathbf{h} \times \mathbf{l} \rangle$ and negate $\mathbf{b}_2$ if it is negative.*

5 Frame Construction Algorithms

The rotations move a frame that already exists. Three further operations build or rebuild one.

5.1 Orient

ORIENT places the turtle at a point and aims its heading and left at two other points. This is the operation that puts a turtle on an atom: the position is the alpha carbon, the heading point is the carbonyl carbon or the beta carbon, and the left point is the nitrogen. The remaining $N - 2$ basis vectors are not determined by the inputs and are completed by Gram-Schmidt from the standard basis [3].

Algorithm 1 ORIENT$(\mathbf{p}, \mathbf{h}, \mathbf{l})$

Require: Position $\mathbf{p}$, a point $\mathbf{h}$ to head toward, a point $\mathbf{l}$ on the left
Ensure: Frame F orthonormal with $\mathbf{b}_0 \parallel \mathbf{h} - \mathbf{p}$ and $\mathbf{b}_1$ in the plane of $\mathbf{b}_0$ and $\mathbf{l} - \mathbf{p}$

1: $\mathbf{b}_0 \leftarrow (\mathbf{h} - \mathbf{p}) / \|\mathbf{h} - \mathbf{p}\|$
2: $\mathbf{v} \leftarrow (\mathbf{l} - \mathbf{p}) - \langle \mathbf{l} - \mathbf{p}, \mathbf{b}_0 \rangle \, \mathbf{b}_0$
3: $\mathbf{b}_1 \leftarrow \mathbf{v} / \|\mathbf{v}\|$
4: **for** $k = 2, \ldots, N - 1$ **do**
5: $\quad \mathbf{v} \leftarrow \mathbf{e}_k - \sum_{m<k} \langle \mathbf{e}_k, \mathbf{b}_m \rangle \, \mathbf{b}_m$
6: $\quad$ **if** $\|\mathbf{v}\| < 10^{-12}$ **then** ▷ $\mathbf{e}_k$ lies in the span of the earlier vectors
7: $\quad\quad$ try $\mathbf{e}_0, \mathbf{e}_1, \ldots$ in turn until one gives $\|\mathbf{v}\| \geq 10^{-12}$
8: $\quad$ **end if**
9: $\quad \mathbf{b}_k \leftarrow \mathbf{v} / \|\mathbf{v}\|$
10: **end for**

The fallback on line 7 is needed. If the heading is aimed along $\mathbf{e}_2$, then at $k = 2$ the projection of $\mathbf{e}_2$ onto $\mathbf{b}_0$ removes all of it and the residual is zero; another standard basis vector must be used instead. The tests include this case. The completion is deterministic, which is worth more than any particular choice of the extra vectors: the same inputs always produce the same frame.

5.2 Orient Toward

ORIENTTOWARD rotates the heading to a given direction $\mathbf{d}$ with a single Givens rotation, and reports the angle it rotated through. In WaveRider this is called with a principal direction from a local PCA, or with the negative time axis (Section 7.3).

Algorithm 2 OrientToward($\mathbf{d}$)

Require: Direction $\mathbf{d} \in \mathbb{R}^N$
Ensure: $\mathbf{b}_0 = \mathbf{d}/\|\mathbf{d}\|$; returns the rotation angle in degrees
if $\|\mathbf{d}\| < 10^{-12}$ **then return** 0
end if
$\mathbf{d} \leftarrow \mathbf{d}/\|\mathbf{d}\|$
$c \leftarrow \langle \mathbf{b}_0, \mathbf{d} \rangle$, clipped to $[-1, 1]$
if $|c| > 1 - 10^{-10}$ **then return** 0
end if ▷ already parallel or antiparallel
$\theta \leftarrow \arccos c$
$\mathbf{q} \leftarrow \mathbf{d} - c\,\mathbf{b}_0$; $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$ ▷ unit component of $\mathbf{d}$ perpendicular to heading
$j \leftarrow \arg\max_{m \geq 1} |\langle \mathbf{b}_m, \mathbf{q} \rangle|$ ▷ basis vector most aligned with $\mathbf{q}$
$\mathbf{b}_j \leftarrow \mathbf{q}$ ▷ temporary: frame is no longer orthonormal
$F \leftarrow G(0, j, \theta)\, F$ ▷ now $\mathbf{b}_0 = c\,\mathbf{b}_0 + \sin\theta\,\mathbf{q} = \mathbf{d}$
Orthonormalize()
return θ in degrees

The rotation is in the plane spanned by the current heading and $\mathbf{d}$, which is the plane of $\mathbf{b}_0$ and $\mathbf{q}$. That plane is not in general spanned by any two basis vectors of the frame, so the algorithm swaps $\mathbf{q}$ into the slot of the basis vector it most resembles, rotates, and then repairs the frame. Because $\mathbf{b}_0$ comes first in the Gram-Schmidt order of Orthonormalize, the repair leaves the new heading exactly $\mathbf{d}$ and adjusts the other vectors. The antiparallel case $c = -1$ returns without rotating; the plane of rotation is undefined there, and the caller who wants the turtle turned around has a whole $(N-1)$-sphere of choices and must make one.

5.3 Orthonormalize

Each rotation renormalizes the two basis vectors it changes but does not restore their orthogonality to each other or to the rest of the frame. Orthonormalize rebuilds the whole frame from a QR decomposition of $F^\top$, whose Q factor is the Gram-Schmidt orthonormalization of the basis vectors in order $0, 1, \ldots, N-1$ [3]. A QR routine is free to return $-\mathbf{q}_k$ in place of $\mathbf{q}_k$, so each column is compared with the basis vector it replaces and negated if the inner product is negative. The result is the orthonormal frame nearest to the drifted one in the sense that $\mathbf{b}_0$ keeps its direction exactly, $\mathbf{b}_1$ keeps its component orthogonal to $\mathbf{b}_0$, and so on down the list.

5.4 Expand Dimension

ExpandDim grows a live turtle from N to $N+1$ dimensions without disturbing anything it has done so far. The position gains a zero coordinate, each existing basis vector gains a zero component, and the new basis vector is the new standard axis $\mathbf{e}_N$.

Algorithm 3 ExpandDim()

Ensure: Turtle in $\mathbb{R}^{N+1}$; returns the index N of the new axis
$\mathbf{p} \leftarrow (\mathbf{p}, 0)$
$F \leftarrow \begin{pmatrix} F & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix}$
$N \leftarrow N + 1$
return $N - 1$

Proposition 5.1. ExpandDim *maps an orthonormal frame to an orthonormal frame, and the original N basis vectors are unchanged in their first N components.*

Proof. The new matrix is block diagonal with orthogonal blocks F and (1), so it is orthogonal. The upper-left block is F. □

The new axis is the only basis vector with a nonzero last component, so every existing rotation and move of the turtle takes place in the original N-dimensional subspace until a rotation involving the new index is issued. The operation is $O(N^2)$ and is a copy, not a recomputation.

6 Numerical Behavior

6.1 Cost

Table 4 gives the cost of each operation. A rotation touches two rows of the frame and costs $O(N)$; this is the property that makes the Givens formulation attractive at large N, since a general rotation matrix applied to the frame would cost $O(N^3)$. The coordinate transforms are one matrix-vector product each.

Table 4: Cost per operation for a turtle in N dimensions.

Operation	**Cost**	**Notes**
Move	$O(N)$	
Rotate and named rotations	$O(N)$	two rows, two norms
ToLocal, ToGlobal	$O(N^2)$	one matrix-vector product
Orient	$O(N^3)$	Gram-Schmidt over $N-2$ vectors
OrientToward	$O(N^3)$	one rotation, one Orthonormalize
Orthonormalize	$O(N^3)$	QR of $F^\top$
ExpandDim	$O(N^2)$	copy into a larger array

6.2 Drift

In exact arithmetic the frame is orthonormal forever. In floating point each rotation introduces rounding of order the machine epsilon, about 2×10^{-16} for double precision, into the two rows it changes. Renormalizing those rows removes the error in their lengths and leaves the error in their mutual angle, and in their angles with the rest of the frame, to accumulate. The accumulation is a random walk, so after M rotations the departure from orthogonality is of order $\sqrt{M}\,\epsilon$ in the typical case and $M\epsilon$ in the worst case. At $M = 10^6$ rotations that is between 10^{-13} and 10^{-10}. `Turtle3D` has built molecules for thirty-six years with this scheme and no orthonormalization step at all, because a molecule takes a few hundred rotations. A long navigation in N dimensions can call Orthonormalize every few thousand steps at a cost that is negligible against a single PCA of the same size.

6.3 Determinism

Every operation is deterministic. Orient completes the frame from the standard basis in a fixed order, OrientToward picks its rotation plane by a deterministic argmax, and Orthonormalize resolves the QR sign ambiguity against the prior frame. No random numbers are drawn anywhere

in the package. A sequence of commands on a given start state produces the same frame on every run, which is a requirement for the reproducibility of anything built on top.

7 Applications

7.1 Building Molecules

The original purpose of the turtle was to build protein structures from internal coordinates. Each atom after the first three is placed by three numbers: the length of the bond that attaches it, the angle that bond makes with the previous one, and the dihedral angle about the previous bond. A turtle standing on the previous atom, heading along the previous bond, with its left vector in the plane of the previous two bonds, places the new atom by

```
roll(dihedral),   yaw(bond_angle),   move(bond_length),
```

and is then standing on the new atom, ready for the next (Remark 4.1). No global coordinates enter at any point. The same three commands build a peptide backbone, a side chain, or a disulfide bridge, and the shape of the molecule is set by the dihedral angles alone once the lengths and angles are fixed at their tabulated values.

`Turtle3D` adds two conventions for proteins. A turtle on an alpha carbon can be in the *backbone* orientation, heading toward the carbonyl carbon with the nitrogen on its left, or the *side-chain* orientation, heading toward the beta carbon with the nitrogen on its left. `orient_at_residue` places the turtle in either orientation from the atoms of a residue, and `bbone_to_schain` and `schain_to_bbone` convert between them with a fixed sequence of four rotations, since the tetrahedral geometry at the alpha carbon is the same for every residue. proteusPy uses this to build a disulfide bond in the turtle's local frame from its five dihedral angles and then transform it into the frame of any residue in the Protein Data Bank [7, 8].

7.2 Navigating an Embedding Space

WaveRider uses `TurtleND` as the moving coordinate system for a walk on a data manifold in $\mathbb{R}^N$ [10]. At each position it takes the k nearest embedding vectors, computes their covariance, and reads off the principal directions and eigenvalues. The frame is aligned so that $\mathbf{b}_0$ is the first principal direction, $\mathbf{b}_1$ the second, and so on; the number d of eigenvalues needed to reach a fixed fraction of the variance is the local intrinsic dimensionality. A step is then taken by bringing the descent direction into the frame with ToLocal, zeroing the components with index d and above, scaling the rest by their eigenvalues, and returning with ToGlobal. The turtle supplies the frame, the transforms, and the rotations that steer between principal directions; the covariance, the thresholds and the objective belong to the walker, which is why they are not in this package. The rest of that method, and its use in estimating intrinsic dimensionality and sizing classifiers, is documented with WaveRider itself.

7.3 Adding a Time Axis

ExpandDim exists for a corpus that has both a semantic embedding and a date. Embedding the text gives a turtle in $\mathbb{R}^N$; appending a standardized fractional year as coordinate N gives a turtle in $\mathbb{R}^{N+1}$ that can be steered in time as easily as in meaning. `orient_in_time` is the convenience call OrientToward$(-\mathbf{e}_N)$. The sign is a convention of the corpus it was written for, whose time coordinate is a z-score that decreases toward the present, so that "forward in time" is the negative direction of the axis. Callers with the opposite convention pass $+\mathbf{e}_N$ to OrientToward directly.

8 Implementation

`turtlend` is a Python package on PyPI with a single runtime dependency, NumPy [4]. It provides three classes. `TurtleND` is the N-dimensional turtle of Section 3; its frame is an $N \times N$ NumPy array of doubles with basis vectors as rows, and ROTATE is implemented as Equation (3) on two rows followed by renormalization. `Turtle3D` is the three-dimensional turtle with its original capitalized property names, its `Vector3D` return types and its residue-building methods, kept intact so that proteusPy code runs unchanged. `Vector3D` is the small vector class `Turtle3D` uses, with helper functions for angles, dihedrals and distances. The three modules total about 1,600 lines including docstrings, of which `TurtleND` is about 500.

The tests cover every operation individually, the 200-step random agreement check of Theorem 4.1, the degenerate cases of ORIENT and ORIENTTOWARD, and the tape. Coverage is held above 80 percent. The package requires Python 3.12 or 3.13, is licensed BSD-3-Clause, and has a documentation site with a frame-conventions page and an API reference at `https://flux-frontiers.github.io/turtlend/`.

9 Conclusion

The turtle is a small object: a point, N unit vectors, and one rotation primitive. Its value is that every operation is expressed in its own frame, so the description of a path, a molecule or a walk on a manifold does not change when the ambient coordinates do. The three-dimensional version has been placing atoms since 1990 and the N-dimensional version is a strict generalization of it, with the 1990 conventions preserved so that the two agree to 10^{-7} after 200 random commands. The whole of the N-dimensional turtle fits in about 500 lines of NumPy.

References

[1] Harold Abelson and Andrea diSessa. *Turtle Geometry: The Computer as a Medium for Exploring Mathematics*. MIT Press, Cambridge, MA, 1981.

[2] Wallace Givens. Computation of plane unitary rotations transforming a general matrix to triangular form. *Journal of the Society for Industrial and Applied Mathematics*, 6(1):26–50, 1958. doi: 10.1137/0106004.

[3] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4th edition, 2013.

[4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[5] Carl O. Pabo and Eric G. Suchanek. Computer-aided model-building strategies for protein design. *Biochemistry*, 25(20):5987–5991, 1986. doi: 10.1021/bi00368a023.

[6] Seymour Papert. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, NY, 1980.

[7] Eric G. Suchanek. proteusPy: A Python package for protein structure and disulfide bond modeling and analysis. *Journal of Open Source Software*, 9(100):6169, 2024. doi: 10.21105/joss.06169.

[8] Eric G. Suchanek. Structural analysis of disulfide bonds in the RCSB protein data bank using proteusPy. In Philip J. Hogg, editor, *Functional Disulphide Bonds: Methods and Protocols*, volume 3016 of *Methods in Molecular Biology*, chapter 2, pages 15–32. Springer US, New York, NY, 2nd edition, 2026. doi: 10.1007/978-1-0716-5158-2_2.

[9] Eric G. Suchanek. turtlend: Orthonormal-frame turtles in 3 and N dimensions. Python package, version 0.1.1, 2026. URL https://github.com/Flux-Frontiers/turtlend.

[10] Eric G. Suchanek. WaveRider: Manifold-aware geometric machine learning. Software, version 0.16.2, 2026.